

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №27

Тема: Алгоритми сортування

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Систематизувати знання про алгоритми сортування, ознайомитися з класифікацією методів сортування та їхніми властивостями (складність, стійкість, використання пам'яті), опанувати ефективні алгоритми — сортування злиттям, пірамідальне сортування та сортування підрахунком — і провести їх порівняльний аналіз із прямими методами та вбудованою функцією `sorted()`.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), бібліотека `matplotlib` (для графіків).

Короткі теоретичні відомості

1. Задача сортування — перестановка елементів послідовності так, щоб вони розташувалися у порядку зростання (спадання) значень ключа. Сортування є основою пошуку, злиття даних, видалення дублікатів, побудови рейтингів і багатьох інших алгоритмів.

2. Класифікація та властивості методів:

- **за складністю:** прямі методи $O(n^2)$ (вибором, вставками, бульбашкове), покращені $O(n \cdot \log^2 n)$ або $O(n^{1.5})$ (Шелла), ефективні $O(n \cdot \log n)$ (злиттям, пірамідальне, швидке), лінійні $O(n + k)$ (підрахунком, порозрядне) — останні не використовують порівнянь;
- **стійкість** — стійке сортування зберігає відносний порядок елементів із однаковими ключами (критично при сортуванні записів за кількома ключами послідовно);
- **«на місці» (in-place)** — потребує $O(1)$ або $O(\log n)$ додаткової пам'яті;
- **адаптивність** — швидше працює на частково впорядкованих даних;
- **внутрішнє / зовнішнє** — дані повністю в оперативній пам'яті або обробляються частинами з диска.

3. Нижня межа для сортувань на порівняннях. Будь-який алгоритм, що вибирає порядок лише порівнянням елементів, у найгіршому випадку потребує щонайменше $\lceil \log_2(n!) \rceil \approx n \cdot \log_2 n$ порівнянь — це впливає з того, що дерево рішень має розрізняти $n!$ перестановок. Тому $O(n \cdot \log n)$ — оптимальна асимптотика для таких методів.

4. Сортування злиттям (Merge Sort). «Розділяй і володарюй»: масив ділиться навпіл, кожна половина сортується рекурсивно, потім дві відсортовані половини **зливаються** в одну за один лінійний прохід (порівнюються найменші невикористані елементи обох частин). Складність у всіх випадках $O(n \cdot \log n)$, стійке, потребує $O(n)$ додаткової пам'яті.

5. Пірамідальне сортування (Heap Sort). Масив перетворюється на **max-піраміду** — двійкове дерево, у якому кожен батько не менший за дітей (для елемента i діти мають індекси

$2i+1$ та $2i+2$). Потім корінь (максимум) обмінюється з останнім елементом, розмір піраміди зменшується на 1, і піраміда відновлюється просіюванням (sift-down). Складність $O(n \log n)$ завжди, виконується «на місці», **нестійке**.

6. Сортування підрахунком (Counting Sort). Для цілих ключів із невеликого діапазону $[0, k]$ підраховують кількість кожного значення в масиві `count[v]`, після чого формують відсортований масив. Складність $O(n + k)$, додаткова пам'ять $O(k)$; ефективно, коли k не набагато більше n . Не є сортуванням на порівняннях, тому нижня межа $n \log n$ на нього не поширюється.

Алгоритм	Найкращий	Середній	Найгірший	Пам'ять	Стійкий
Вибором	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	ні
Вставками	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	так
Бульбашкове	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$	так
Шелла	$O(n \log n)$	залежить від кроків	$O(n^2)$	$O(1)$	ні
Швидке	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	ні
Злиттям	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	так
Пірамідальне	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	ні
Підрахунком	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	так

$\backslash$ — для оптимізованої версії з прапорцем раннього завершення.* Вбудована функція Python `sorted()` (і метод `list.sort()`) використовує алгоритм **Timsort** — гібрид злиття та вставок, адаптивний, стійкий, із найгіршою складністю $O(n \log n)$.

Хід роботи

Завдання 1. Ручне сортування злиттям

Виконати вручну сортування злиттям масиву $[38, 27, 43, 3, 9, 82, 10]$ із зображенням дерева розбиття та злиття.

Порядок виконання:

1. Намалюйте дерево розбиття: поділіть масив навпіл (7 елементів $\rightarrow 3 + 4$), і так далі до підмасивів довжиною 1 ; запишіть, які підмасиви утворилися на кожному рівні.
2. Виконайте злиття знизу вгору: на кожному рівні запишіть пари відсортованих підмасивів і результат їх злиття; для злиття двох підмасивів покажіть порівняння початкових елементів.
3. Для останнього злиття ($[3, 27, 38, 43]$ і $[9, 10, 82]$) складіть таблицю «крок — елемент лівого — елемент правого — що записано — результат» і порахуйте кількість порівнянь.
4. Визначте глибину дерева розбиття та порівняйте її з $\lceil \log_2 7 \rceil$.
5. Поясніть, чому злиття є стійким, якщо при рівних елементах першим записується елемент лівої частини.

Завдання 2. Ручна побудова піраміди та пірамідальне сортування

Побудувати вручну тах-піраміду з масиву $[4, 10, 3, 5, 1, 8]$ і виконати два перших кроки вилучення максимуму.

Порядок виконання:

1. Зобразіть масив у вигляді двійкового дерева (діти елемента i — індекси $2i+1$ та $2i+2$) та визначте, які елементи не є листками (для $n = 6$ — індекси 0, 1, 2).
2. Побудуйте піраміду просіюванням знизу вгору: починайте з елемента $n // 2 - 1 = 2$ і рухайтесь до кореня; на кожному кроці порівняйте батька з більшим із дітей і за потреби обміняйте їх.
3. Запишіть масив після побудови піраміди й перевірте умову піраміди для кожного батька.
4. Виконайте перший крок сортування: обміняйте корінь з останнім елементом піраміди, зменшіть розмір піраміди й відновіть її просіюванням кореня; запишіть масив.
5. Виконайте другий крок аналогічно; запишіть, які елементи вже стоять на своїх остаточних місцях.

Завдання 3. Програмна реалізація ефективних алгоритмів сортування

Реалізувати мовою Python сортування злиттям, пірамідальне сортування та сортування підрахунком; перевірити коректність порівнянням із `sorted()` та підрахувати кількість порівнянь.

Порядок виконання:

1. Напишіть функцію `merge_sort(arr)`, що повертає новий відсортований список і кількість порівнянь; допоміжну функцію `merge(left, right)` реалізуйте двома індексами (при `left[i] <= right[j]` беріть елемент лівої частини).
2. Напишіть функцію `heap_sort(arr)` із допоміжною функцією `sift_down(a, n, i)`; спочатку побудуйте піраміду, потім виконайте $n - 1$ обмінів із відновленням піраміди.
3. Напишіть функцію `counting_sort(arr, k)`, що сортує цілі числа з діапазону $[0, k]$ за допомогою масиву лічильників.
4. Перевірте всі три функції на тестових масивах із завдань 1, 2 (результати збігаються з ручними), а також на порожньому масиві, масиві з одного елемента та масиві з дублікатами.
5. Перевірте стійкість: відсортуйте пари (ключ, мітка) за ключем злиттям (`merge_sort_key`), пірамідою (`heap_sort_pairs`) та функцією `sorted(key=...)` і порівняйте порядок міток у рівних ключів.

Приклад реалізації мовою Python:

python

```
def merge(left, right, cnt):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        cnt[0] += 1
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

def merge_sort(arr, cnt=None):
    top = cnt is None
    cnt = [0] if top else cnt
    if len(arr) > 1:
        mid = len(arr) // 2
```

```

    arr = merge(merge_sort(arr[:mid], cnt), merge_sort(arr[mid:], cnt), cnt)
    return (arr, cnt[0]) if top else arr

def sift_down(a, n, i, cnt):
    while True:
        largest, l, r = i, 2 * i + 1, 2 * i + 2
        if l < n:
            cnt[0] += 1
            if a[l] > a[largest]:
                largest = l
        if r < n:
            cnt[0] += 1
            if a[r] > a[largest]:
                largest = r
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest

def heap_sort(arr):
    a, n, cnt = arr.copy(), len(arr), [0]
    for i in range(n // 2 - 1, -1, -1):
        sift_down(a, n, i, cnt)
    heap_state = a.copy()
    for end in range(n - 1, 0, -1):
        a[0], a[end] = a[end], a[0]
        sift_down(a, end, 0, cnt)
    return a, cnt[0], heap_state

def counting_sort(arr, k):
    count = [0] * (k + 1)
    for v in arr:
        count[v] += 1
    result = []
    for v in range(k + 1):
        result.extend([v] * count[v])
    return result

def merge_sort_key(arr, key):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left, right = merge_sort_key(arr[:mid], key), merge_sort_key(arr[mid:], key)
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

def heap_sort_pairs(pairs):
    a, n = pairs.copy(), len(pairs)

    def sift(i, size):
        while True:
            m, l, r = i, 2 * i + 1, 2 * i + 2
            if l < size and a[l][0] > a[m][0]:
                m = l
            if r < size and a[r][0] > a[m][0]:
                m = r
            if m == i:
                return

```

```

        a[i], a[m] = a[m], a[i]
        i = m

    for i in range(n // 2 - 1, -1, -1):
        sift(i, n)
    for end in range(n - 1, 0, -1):
        a[0], a[end] = a[end], a[0]
        sift(0, end)
    return a

t1 = [38, 27, 43, 3, 9, 82, 10]
res, c = merge_sort(t1)
print("Злиттям:", res, ", порівнянь:", c)
res, c, heap = heap_sort([4, 10, 3, 5, 1, 8])
print("Піраміда після побудови:", heap)
print("Пірамідальне:", res, ", порівнянь:", c)
print("Підрахунком:", counting_sort([4, 2, 2, 8, 3, 3, 1, 0, 8], 8))

for t in ([], [7], [5, 5, 5], [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]):
    ok = merge_sort(t)[0] == heap_sort(t)[0] == sorted(t)
    print(f"{t!s:<32} коректно: {ok}")

pairs = [(2, "a"), (1, "b"), (2, "c"), (1, "d"), (2, "e")]
print("Злиття за ключем:", merge_sort_key(pairs, lambda p: p[0]))
print("sorted(key=...):", sorted(pairs, key=lambda p: p[0]))
print("Пірамідальне за ключем:", [p for p in heap_sort_pairs(pairs)])

```

Результат виконання програми:

```

Злиттям: [3, 9, 10, 27, 38, 43, 82] , порівнянь: 13
Піраміда після побудови: [10, 5, 8, 4, 1, 3]
Пірамідальне: [1, 3, 4, 5, 8, 10] , порівнянь: 15
Підрахунком: [0, 1, 2, 2, 3, 3, 4, 8, 8]
[] коректно: True
[7] коректно: True
[5, 5, 5] коректно: True
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0] коректно: True
Злиття за ключем: [(1, 'b'), (1, 'd'), (2, 'a'), (2, 'c'), (2, 'e')]
sorted(key=...): [(1, 'b'), (1, 'd'), (2, 'a'), (2, 'c'), (2, 'e')]
Пірамідальне за ключем: [(1, 'b'), (1, 'd'), (2, 'c'), (2, 'e'), (2, 'a')]

```

Висновок за результатами: результати програм збігаються з ручними трасуваннями: злиттям отримано [3, 9, 10, 27, 38, 43, 82], побудована піраміда має вигляд [10, 5, 8, 4, 1, 3], де кожен батько не менший за дітей. Сортування підрахунком обробляє дублікати без жодного порівняння елементів. Функція sorted() і злиття зберігають порядок міток для рівних ключів (стійкі), тоді як піраміда його порушує — це наочно підтверджує, що пірамідальне сортування нестійке.

Завдання 4. Порівняльне дослідження алгоритмів сортування

Експериментально порівняти час роботи та кількість порівнянь прямих і ефективних методів на випадкових даних зростаючого розміру та підтвердити асимптотику $O(n^2)$ та $O(n \cdot \log n)$.

Порядок виконання:

1. Використайте функції `insertion_sort` та `bubble_sort` (з практичних робіт №8 і №13) для порівняння з методами завдання 3. Згенеруйте випадкові масиви розмірів $n = 1000, 2000, 4000, 8000$.
2. Для кожного методу й розміру виміряйте час виконання за допомогою `time.perf_counter()` (для надійності візьміть найкращий результат із 3 запусків) і занесіть у таблицю « n — вставки — злиття — піраміда — sorted».
3. Виміряйте кількість порівнянь вставок, злиття та піраміди для $n = 1000, 2000, 4000, 8000$ і порівняйте з теоретичною оцінкою $n \cdot \log_2 n$ (для злиття) та $n^2/4$ (для вставок).

4. Для кожної пари сусідніх розмірів обчисліть, у скільки разів зріс час: для $O(n^2)$ очікується ≈ 4 , для $O(n \log n)$ – $\approx 2,2$.
5. Побудуйте графік залежності часу від n для вставок, злиття та `sorted()` і зробіть висновок про практичну доцільність кожного методу.
6. Порівняйте з `sorted()` час сортування масиву з $n = 100\,000$ елементів із діапазону $0 \dots 100$ для функцій `counting_sort` та `merge_sort`; поясніть результат.

Приклад реалізації мовою Python:

python

```
import math
import random
import time
import matplotlib.pyplot as plt

random.seed(42)

def insertion_sort(arr):
    a, c = arr.copy(), 0
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0:
            c += 1
            if a[j] > key:
                a[j + 1] = a[j]
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, c

def best_time(func, arr, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        func(arr)
        best = min(best, time.perf_counter() - t0)
    return best

sizes = [1000, 2000, 4000, 8000]
times = {"вставки": [], "злиття": [], "піраміда": [], "sorted": []}
cmprs = {"вставки": [], "злиття": [], "піраміда": []}
print("n\t вставки(c)\t злиття(c)\t піраміда(c)\t sorted(c)")
for n in sizes:
    arr = [random.randint(0, 10**6) for _ in range(n)]
    t = [best_time(insertion_sort, arr), best_time(merge_sort, arr),
         best_time(heap_sort, arr), best_time(sorted, arr)]
    for name, v in zip(times, t):
        times[name].append(v)
    cmprs["вставки"].append(insertion_sort(arr)[1])
    cmprs["злиття"].append(merge_sort(arr)[1])
    cmprs["піраміда"].append(heap_sort(arr)[1])
    print(f"{n:>6}\t {t[0]:>9.4f}\t {t[1]:>9.4f}\t {t[2]:>11.4f}\t {t[3]:>9.6f}")

print("\n\t\t\t\t\t порівнянь: вставки\t злиття\t піраміда\t n*log2(n)\t n^2/4")
for i, n in enumerate(sizes):
    print(f"{n:>6}\t {cmprs['вставки'][i]:>17}\t {cmprs['злиття'][i]:>9}\t {cmprs['піраміда'][i]:>10}\t "
          f"{n * math.log2(n):>12.0f}\t {n * n // 4:>8}")

print("\nЗростання часу при подвоєнні n (вставки / злиття):")
for i in range(1, len(sizes)):
    print(f"{sizes[i - 1]:>5}\t -> {sizes[i]:>5}: x{times['вставки'][i] / times['вставки'][i - 1]:.1f}\t / "
          f"x{times['злиття'][i] / times['злиття'][i - 1]:.1f}")
```

```

        f"x{times['злиття'][i] / times['злиття'][i - 1]:.1f}")

big = [random.randint(0, 100) for _ in range(100000)]
print(f"\nn = 100000, діапазон 0..100: підрахунок {best_time(lambda a: counting_sort(a,
100), big):.4f} с, "
      f"злиття {best_time(merge_sort, big):.4f} с, sorted {best_time(sorted, big):.4f}
с")

plt.figure(figsize=(7, 4.2))
for name in ("вставки", "злиття", "sorted"):
    plt.plot(sizes, times[name], "o-", label=name)
plt.xlabel("розмір масиву n")
plt.ylabel("час, с")
plt.title("Час сортування залежно від розміру масиву")
plt.grid(alpha=0.4)
plt.legend()
plt.show()

```

Результат виконання програми:

n	вставки(с)	злиття(с)	піраміда(с)	sorted(с)
1000	0.0165	0.0015	0.0017	0.000072
2000	0.0693	0.0033	0.0040	0.000193
4000	0.2846	0.0077	0.0085	0.000480
8000	1.1477	0.0167	0.0188	0.001071

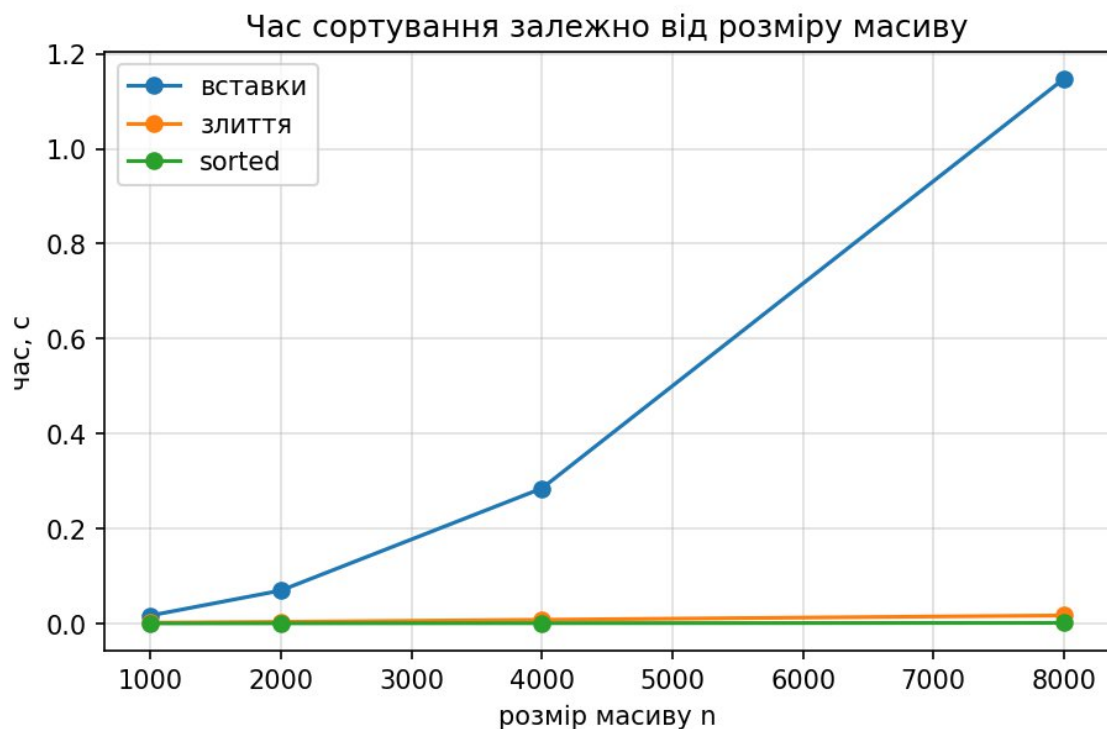
n	порівнянь:	вставки	злиття	піраміда	$n \cdot \log_2(n)$	$n^2/4$
1000		240414	8698	16894	9966	250000
2000		1006984	19451	37702	21932	1000000
4000		3970139	42804	83308	47863	4000000
8000		16030552	93704	182865	103726	16000000

Зростання часу при подвоєнні n (вставки / злиття):

1000 -> 2000: x4.2 / x2.2
 2000 -> 4000: x4.1 / x2.3
 4000 -> 8000: x4.0 / x2.2

n = 100000, діапазон 0..100: підрахунок 0.0033 с, злиття 0.2317 с, sorted 0.0114 с

Графік порівняння:



Висновок за результатами: кількість порівнянь сортування вставками для випадкових даних близька до $n^2/4$ і зростає приблизно у 4 рази при подвоєнні n , тоді як у злиття та піраміди вона наближається до $n \cdot \log_2 n$ (зростає лише у $\approx 2,1$ раза). Тому на масиві з 8000 елементів злиття виконує приблизно в 171 раз менше порівнянь, ніж вставки. Піраміда робить більше порівнянь, ніж злиття, але не потребує додаткової пам'яті. Вбудована функція `sorted()` значно швидша за ручні реалізації злиття й піраміди, бо написана на мові C. Сортування підрахунком при малому діапазоні значень (0...100) виявилось швидшим навіть за `sorted()`, оскільки не виконує порівнянь, а за один прохід підраховує елементи (абсолютні значення часу залежать від комп'ютера).

Контрольні питання

1. За якими ознаками класифікують алгоритми сортування? Що таке стійкість і чому вона важлива?
2. Чому нижня межа складності сортувань на порівняннях дорівнює $n \cdot \log n$?
3. Опишіть сортування злиттям. Чому воно стійке й яку додаткову пам'ять потребує?
4. Що таке піраміда (heap)? Як побудувати її з довільного масиву та як працює просіювання (sift-down)?
5. Чому пірамідаліне сортування нестійке, але виконується «на місці»?
6. Коли доцільно використовувати сортування підрахунком? Чому на нього не поширюється нижня межа $n \cdot \log n$?
7. Який алгоритм використовує вбудована функція `sorted()` у Python? Назвіть його властивості.
8. Як вибрати метод сортування для таких задач: масив майже відсортований; потрібна стійкість; обмежена пам'ять; числа від 0 до 100?

Зміст звіту

1. Тема, мета роботи.
2. Ручне трасування сортування злиттям та побудова піраміди (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Таблиці часу та кількості порівнянь, графік залежності часу від n .
5. Висновки.